ITA0612-Machine Learning

CO4 - CODING CHALLENGE

Divyesh S P (192424147)

1.Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbours and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Code:

```
import math

# Small dataset: [Height, Weight]
X = [[5,50], [5.5,55], [6,60], [6.5,65], [7,70], [5.2,52], [6.2,62], [6.8,68]]
Y = ['A','A','A','B','B','A','B','B']

# Test data
test = [[5.8,58], [6.7,67], [5.3,53], [6.4,64]]
actual = ['A','B','A','B']

k = int(input("Enter K: "))

def knn(x):
    # Euclidean distance
    d = [(math.sqrt(sum((x[j]-X[i][j])**2 for j in range(2))), Y[i])
         for i in range(len(X))]
    d.sort()

    # Majority voting
    labels = [label for _, label in d[:k]]
    return max(set(labels), key=labels.count)

pred = [knn(x) for x in test]

print("\nPredicted Results:")
for i in range(len(test)):
    print(f"Test {i+1}: Actual={actual[i]}, Predicted={pred[i]}")

accuracy = sum(p == a for p, a in zip(pred, actual)) / len(actual) * 100
print(f"\nAccuracy = {accuracy:.2f}%")
```

Output:

```
Enter K: 4

Predicted Results:
Test 1: Actual=A, Predicted=A
Test 2: Actual=B, Predicted=B
Test 3: Actual=A, Predicted=A
Test 4: Actual=B, Predicted=B

Accuracy = 100.00%
```

2. Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Code:

```
import math

# Training data: Hours studied
X = [1,2,3,4,5,6,7,8]
Y = [0,0,0,0,1,1,1,1]

# Test data
test = [2.5,4.5,5.5,7.5]
actual = [0,0,1,1]

lr = float(input("Enter learning rate: "))
epochs = int(input("Enter epochs: "))

w, b = 0, 0

def sigmoid(z):
    return 1 / (1 + math.exp(-z))
```

```
# Gradient Descent
for epoch in range(epochs):
    dw = db = 0

    for x, y in zip(X, Y):
        p = sigmoid(w*x + b)
        dw += (p-y)*x
        db += p-y

    w -= lr * dw / len(X)
    b -= lr * db / len(X)

    if (epoch+1) % 100 == 0:
        loss = sum(
            -(y*math.log(max(sigmoid(w*x+b),1e-10)) +
              (1-y)*math.log(max(1-sigmoid(w*x+b),1e-10)))
            for x,y in zip(X,Y)
        ) / len(X)
        print(f"Epoch {epoch+1}: Loss={loss:.4f}")

# Prediction
print("\nResults:")
pred = []

for x, y in zip(test, actual):
    p = sigmoid(w*x+b)
    c = 1 if p >= 0.5 else 0
    pred.append(c)
    print(f"Input={x}, Sigmoid={p:.4f}, Actual={y}, Predicted={c}")

accuracy = sum(p == y for p,y in zip(pred,actual)) / len(actual) * 100
print(f"\nAccuracy = {accuracy:.2f}%")
```

Output:

```
Enter learning rate: 0.3
Enter epochs: 1000
Epoch 100: Loss=0.2626
Epoch 200: Loss=0.1910
Epoch 300: Loss=0.1585
Epoch 400: Loss=0.1391
Epoch 500: Loss=0.1257
Epoch 600: Loss=0.1157
Epoch 700: Loss=0.1079
Epoch 800: Loss=0.1015
Epoch 900: Loss=0.0961
Epoch 1000: Loss=0.0915

Results:
Input=2.5, Sigmoid=0.0202, Actual=0, Predicted=0
Input=4.5, Sigmoid=0.5476, Actual=0, Predicted=1
Input=5.5, Sigmoid=0.9027, Actual=1, Predicted=1
Input=7.5, Sigmoid=0.9982, Actual=1, Predicted=1

Accuracy = 75.00%
```